

Full Length Article
LSYS

ST-Align: A time series and text alignment framework for cross-subject multivariate time series classification

Zhenghuang Wu^{a,1}, Tao Zhang^{a,1}, Ke Li^a, Yuangan Li^{a,*}

^a School of Aeronautic Science and Engineering, Beihang University, Beijing, China

ARTICLE INFO

Keywords:

Multivariate time series classification
Large language model (LLM)
Cross-modal alignment

ABSTRACT

Multivariate Time Series Classification (MTSC) aims to discern temporal dynamics among variables to classify time series. However, existing MTSC methods predominantly focus on unimodal data, thereby neglecting the rich semantic information embedded within the associated text labels. This limitation hampers the ability to comprehend complex patterns and generalize effectively, ultimately resulting in suboptimal performance in real-world cross-subject scenarios. To address these issues, we propose a time series and text alignment framework (ST-Align) for cross-subject multivariate time series classification. ST-Align leverages the semantic alignment between time series and text labels to improve classification accuracy in semantic spaces, while harnessing the prior knowledge of a large language model (LLM). Specifically, we introduce a two-stage alignment approach. In the first stage, a fine-grained token alignment is employed to capture local information. Inspired by contrastive learning, the captured information is then contrasted with the original text labels to achieve a hybrid Token-Prototype alignment. Extensive experiments on four cross-subject datasets demonstrate that ST-Align achieves state-of-the-art (SOTA) performance on unseen datasets. Moreover, comparative analyses between models with and without the LLM demonstrate their effectiveness, achieving average accuracies of 86.3% and 88.4%, respectively.

1. Introduction

Time series analysis plays a critical role in a variety of fields, including climate modeling (Mudelsee, 2019), traffic management (Huang et al., 2022), healthcare monitoring (C. C. Liu et al., 2024), and human action recognition (Wang et al., 2024c). Particularly, multivariate time series classification (MTSC), which aims to assign labels to time series, is challenging yet important in most real-world scenarios.

Traditionally, these tasks have predominantly relied on classical signal processing techniques, such as frequency analysis and decomposition-based methods. In recent years, deep learning approaches, including Convolutional Neural Networks (CNNs) (Zhao et al., 2017), Long Short-Term Memory networks (LSTMs) (Lindemann et al., 2021), Graph Neural Networks (GNNs) (Zheng et al., 2024), and Transformers (Wen et al., 2022), have fundamentally transformed this field. These methods have proven highly effective in extracting meaningful patterns from time series data, thus becoming the predominant

approaches for time series analysis across various domains. Despite their distinct architectural predispositions for time series modeling, these methods collectively adhere to a fundamental strategy: segmenting the time series into discrete temporal points and subsequently elucidating the intricate inter-dependencies among these points. However, these methods often overly focus on the training data, resulting in poor model performance when confronted with unseen data. This limitation is particularly salient in real-world applications that demand cross-subject prediction.

In recent years, inspired by the groundbreaking advancements of large models in the fields of text and image processing (Guo et al., 2024), researchers have begun to explore the potential of leveraging the prior knowledge of large models to enhance the performance of time series signal analysis (Sun et al., 2024). Presently, there are primarily two approaches to achieve the integration of Time Series and Large Language Model (TS+LLM): Model-Centric and TS-Centric. The Model-Centric approach involves training a foundational large model

* Corresponding author.

E-mail address: liyangan@buaa.edu.cn (Y. Li).

¹ Equal Contribution.

from scratch using extensive time series data, akin to the training methodology of BERT, followed by fine-tuning it for various downstream tasks (Garza et al., 2024; Liu et al., 2024a; Ma et al., 2024). In contrast, the TS-Centric approach focuses on bridging the modality gap and harnessing the prior knowledge of pre-trained models (C. C. Wang et al., 2024; Wang et al., 2024b; Wang et al., 2025).

Considering the limitations of computational resources and dataset scales, the majority of researchers have gravitated towards the latter approach. For instance, ETP (C. C. Liu et al., 2023) employs contrastive learning-based pre-training to align ECG signals with textual reports. TEST (Sun et al., 2024) and DECA-GNN (Hu et al., 2025) utilize contrastive learning to generate hierarchically aligned time series embeddings to match text embeddings. These alignment methods, primarily based on the principle of contrastive learning, aim to increase the similarity between different modalities. Additionally, GPT4TS (T. T. Zhou et al., 2023), Time-LLM (Jin et al., 2024), and ST-LLM (Liu et al., 2024) adopt LLMs as backbone networks following time series embedding layers. They directly encode time series signals and subsequently feed them into the LLMs, achieving end-to-end training.

LLM-based methods have demonstrated effectiveness, yet several bottlenecks remain when applying LLMs to time series analysis (Jiang et al., 2024). First, capturing temporal dependencies and channel-specific features is crucial for accurate modeling. Second, there is a mismatch between numerical time series data and the semantic space of LLMs, and relying on a single alignment method hinders their ability to effectively process such data. Third, many existing time series datasets lack corresponding text data, making it challenging to obtain alignment anchors. These challenges underscore the need for more efficient approaches for MTSC based on LLMs (Tao et al., 2024).

To this end, we propose a time series and text alignment framework for cross-subject MTSC, called ST-Align. Considering the model architecture, we first employ a two-branch encoder, comprising a patch convolution and a multi-head transformer branch, which can extract local-global representations from raw time series. To better align these representations with the semantic space of LLMs, we introduce a Signal-Text Alignment Module with two hierarchical levels: the token-level and the prototype-level. Specifically, we leverage a pre-trained LLM to generate textual descriptions for elementary time series labels, which serve as alignment anchors. The token-level alignment mechanism

enables fine-grained correspondence and synchronization across the two modalities, while the prototype-level alignment effectively mitigates divergence between the temporal and semantic spaces. Owing to this alignment, the pre-trained LLM is capable of more accurately comprehending the input time series signals, integrating additional prior knowledge, and consequently demonstrating enhanced robustness when encountering unseen data. As shown in Fig. 1, ST-Align provides a new solution paradigm for MTSC. Our contributions are as follows:

- We propose a novel MTSC architecture named ST-Align, which consists of a time series encoding module, a signal-text alignment module, and a pre-trained LLM. ST-Align can efficiently leverage the semantic information of time series labels to harness the prior knowledge of the large language model.
- We adopt a hybrid alignment paradigm that integrates fine-grained token-level alignment with prototype-level semantic clustering, forming a two-stage alignment framework of “micro lexical matching—macro semantic integration.”
- We conducted cross-subject prediction experiments on four MTSC datasets from distinct domains. Results show that ST-Align achieves excellent performance on unseen datasets. Furthermore, we compared the performance of models with and without the LLM: our method outperforms SOTA baselines, with average accuracies of 86.3% and 88.4% respectively, fully demonstrating the effectiveness of the hybrid alignment method.

The rest of this paper is organized as follows: Section 2 provides a comprehensive overview of related work in the field. In Section 3, we present our feature representation, two-stage alignment method, and cross-subject recognition in detail. Section 4 describes the experiments conducted to validate our approach, comparing it with other state-of-the-art (SOTA) baselines. Finally, Section 5 summarizes the main contributions of the paper and discusses potential future research directions.

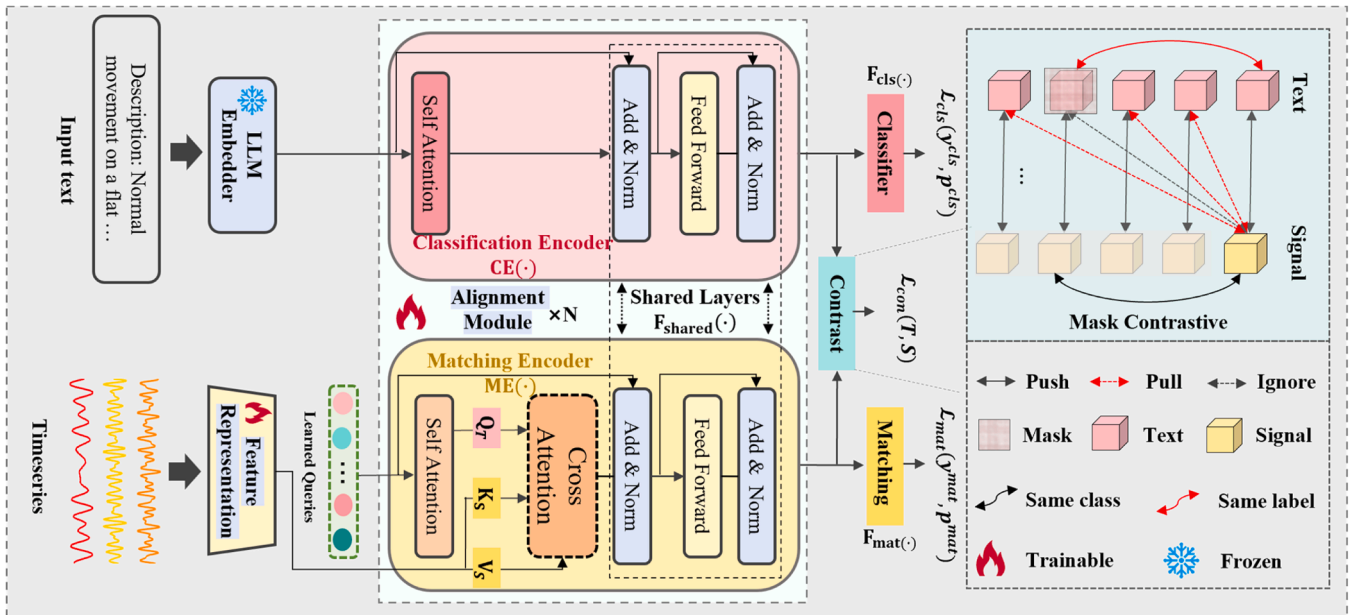


Fig. 1. Overall structure of the proposed ST-Align (w/o LLM), consisting of three modules. (1) LLM Embedder: converting text labels into numerical vectors representing their semantic content; (2) feature Representation: Extracting both global and local information from multivariate time series; (3) alignment Module: achieving hybrid token-prototype alignment.

2. Related work

2.1. Contrastive learning for time series representation

Contrastive learning, a type of self-supervised learning method, has garnered considerable attention in the realm of time series representation learning in recent years (Qian et al., 2025). It aims to learn effective data representations by mapping similar samples to nearby positions in the embedding space and dissimilar samples to distant positions. SimCLR (Chen et al., 2020) originally designed for image representation learning, applies a series of random data augmentations (e.g., random cropping, jittering, rotation) to generate positive sample pairs, optimizing the network via the NT-Xent loss function. MoCo (He et al., 2020) introduces a large dynamic dictionary based on a queue and a moving-average encoder, which enhances small-batch training by maintaining diversity and consistency. TimeCLR (Yang et al., 2022), combining the strengths of SimCLR and InceptionTime, proposes a DTW data augmentation method that not only generates phase shifts and amplitude changes but also retains the structure and feature information of time series to address the Time Series Classification (TSC) problem. TS2Vec (Yue et al., 2022) employs multi-scale contrastive learning to capture hierarchical features, providing robust per-timestamp representations. Beyond the singular time domain, contrastive learning has also extended to the frequency domain and text contrastive learning (Eldele et al., 2021).

However, despite the significant potential of contrastive learning in time series representation, several challenges remain Middlehurst et al. (2024); Mohammadi Foumani et al. (2024). First, defining positive and negative pairs is non-trivial due to the inherent temporal dependencies in time series data, as improper construction can lead to suboptimal learning. Moreover, reliance on data augmentation techniques such as time-masking and jittering may not always generate realistic or semantically meaningful time series variations, potentially undermining the learning process. Meanwhile, contrastive learning methods that leverage textual semantic information in MTSC are still relatively rare.

2.2. LLM and time series: bridging the gap

Large Language Model (LLM) holds significant potential for the analysis of time series data, with applications spanning various domains. (Zhang et al., 2024b) provide a comprehensive review of existing methodologies that leverage LLMs for processing time series data. However, despite their versatility, LLMs are inherently designed for sequential text data rather than heterogeneous numerical time series. Alignment methods are a key aspect of LLM-based approaches (B. B. Zhang et al., 2024). These methods learn a separate embedding for time series and align this embedding with the semantic space of language models. TENT (Zhou et al., 2023b) aligns text embeddings with IoT sensor signals in a unified semantic feature space using contrastive learning, customized descriptions, and soft prompt design. Time-LLM (Jin et al., 2024) employs multi-head cross-attention for patch reprogramming to bridge the gap between text and time series representations. JoLT (Wagner et al., 2020) utilizes the Querying Transformer (Q-Former (Li et al., 2023)) to jointly learn desired representations from pre-trained time series and text. Instruct Time (Cheng et al., 2024) transforms time series classification into direct label generation using LLMs, integrating an aligned projector layer implemented via a multilayer perceptron (MLP) to seamlessly bridge the modality gap. In addition to contrastive loss functions, other loss functions (Han et al., 2024; Qiu et al., 2023) are also employed to optimize the similarity matching between time series and text embeddings. However, these methods are overly simplistic in the alignment process between time series and text, and the text acquisition process is excessively complex. To address these issues, we have simplified the text acquisition process by focusing solely on time series labels and emphasizing the alignment methods between the two modalities to achieve a hybrid alignment of both coarse and fine

granularities.

3. Method

3.1. Overview of proposed framework

In this work, we propose ST-Align, a novel signal-text pre-training method built upon a frozen pre-trained LLM. The aim of this work is to develop a base instructional model with reasoning and generalization capabilities to address the challenges in cross-subject time series classification. Our ST-Align comprises three core components: 1) feature representation, 2) an alignment module, and 3) a pre-trained LLM. These three core modules cooperate in a two-stage training.

During the Signal-Text Alignment stage, in order to bridge the modality gap, we design two transformer sub-modules sharing identical layers except for self-attention: 1) a Classification Encoder (CE) interacts with text embeddings to amplify the semantic differences between distinct text labels; 2) a Matching Encoder (ME) fuses text and time series embeddings via a cross-attention module, as shown in Fig. 1. The integration of text semantics enhances the latent features extraction from time series and simultaneously boosts the generalization capabilities of models. To address the limitations in the semantics of the given text and the missing text labels within the reasoning process, we design the soft queries as input to the ME. To enable efficient alignment while maintaining task specificity, the classification and matching encoders adopt a parameter-sharing strategy that retains all components except for their respective self-attention (SA) and cross-attention (CA) layers.

In the LLM fine-tuning stage, we introduce an adapter module and a classification head, as illustrated in Fig. 3. The adapter serves as a bridge to map the feature space derived from the alignment module into the semantic embedding space compatible with the pre-trained LLM, while the classification head preserves the feature-label correspondence for task-specific prediction. Critically, the backbone LLM and alignment module remain frozen during training, a design that ensures parameter-efficient optimization. The key parameters used in our method are listed in Table 1.

3.2. Feature representation

3.2.1. Global feature extraction

The time series encoder, comprising global and local components to capture long-range dependencies and fine-grained details respectively, presents its global architecture in Fig. 2(a). We use a vanilla Transformer

Table 1
Parameters.

Parameter	Meaning
B	Batch size
L	Length of time series sequence
I	Length of patches processed from time series
C	Number of channels
D	Hidden dimension of latent space
$G(\cdot)$	Global Encoder, Transformer blocks
$L(\cdot)$	Local Encoder, 1D-convolutional network
$E(\cdot)$	Text embedding layer, BERT-base
x_{global}	Global representation of time series
x_{local}	Local representation of time series
x_{text}	Text embedding vectors
$CE(\cdot)$	Classification Encoder in Alignment Module
$ME(\cdot)$	Matching Encoder in Alignment Module
SA_{mat}	Self-attention layer in Matching Encoder
SA_{cls}	Self-attention layer in Classification Encoder
q	Learnable queries
L_q	Length of learnable queries
$\mathcal{L}_{mat}$	Token-level alignment/matching loss
$\mathcal{L}_{cls}$	Prototype-level classification loss
$\mathcal{L}_{con}$	Contrastive loss for signal-text semantic alignment
$\mathcal{L}_{pre}$	Cross-entropy loss for downstream fine-tuning

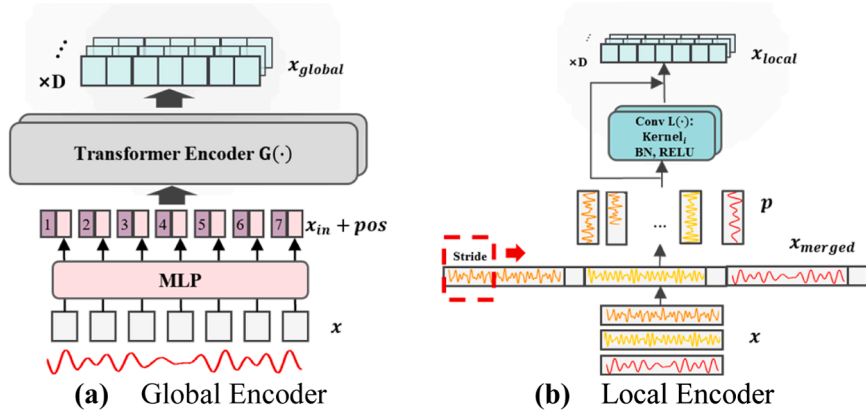


Fig. 2. Architecture of the proposed time series feature representation module: (a) employing a linear projection (MLP) to map inputs to the latent space, followed by a transformer encoder to obtain fine-grained features; (b) utilizing PatchConv with different kernel sizes to obtain multi-scale features.

encoder composed of a stack of multiple identical layers, which maps the original sequence to latent representations. Each layer consists of a multi head self-attention mechanism and a feed-forward network (FFN). The series are first mapped to the Transformer latent space of dimension D via a trainable linear projection, resulting in $x_{in} \in \mathbb{R}^{B \times L \times D}$, where B is the batch size and L is the length of time series. Additionally, the absolute position encoding $pos \in \mathbb{R}^{B \times L \times D}$ is also applied to obtain positional information for time series, which is calculated using the sine and cosine functions of different frequencies in Eq. (1).

$$\begin{aligned} pos_{i,2k-1}^{(b)} &= \sin(i/10000^{(2k-1)/d}) \\ pos_{i,2k}^{(b)} &= \cos(i/10000^{2k/d}) \end{aligned} \quad (1)$$

where $i = 1, \dots, n$ is the position of sequence, $k = 1, \dots, dis$ is the feature dimension index and b represents the index of the specific sample within the current batch.

Then it will be incorporated directly into the input embeddings as shown in Eq. (2). Finally, the new embeddings x'_{in} are fed into a module with several Transformer Blocks $G(\cdot)$ to generate the global representation $x_{global} \in \mathbb{R}^{B \times L \times D}$, as shown in Eq. (3).

$$x'_{in} = x_{in} + pos \quad (2)$$

$$x_{global} = G(x'_{in}) \quad (3)$$

3.2.2. Local feature extraction

Inspired by the architecture of ResNet (He et al., 2015), we have developed a specialized module called PatchConv aimed at extracting local features from multivariate time series, as depicted in Fig. 2(b). Given the input multivariate time series $x_{in} \in \mathbb{R}^{B \times L \times C}$, where C denotes the number of channels, replication padding is applied to the second dimension $x_{pad} \in \mathbb{R}^{B \times (L+pad) \times C}$ to ensure the sliding window does not exceed the boundaries. Then, the channel and sequence dimensions are merged into $x_{merged} \in \mathbb{R}^{B \times ((L+pad) \times C)}$, which will be divided into several patches with non-overlapping stride P . Following the patching process, we obtain a sequence of patches $p = \{p_i \in \mathbb{R}^{B \times P} | i = 1, \dots, I\}$, where I denotes the sequence length of patches. Through 1D-convolutional network blocks $L(\cdot)$ (including BatchNorm, ReLU and Shortcut), as shown in Eq. (4), the patches p are then projected into the local representation $x_{local} \in \mathbb{R}^{B \times I \times P}$.

$$\begin{aligned} x_{local} &= L(p) \\ &= \text{ReLU}(\text{BN}(\text{Conv}(p))) + p \end{aligned} \quad (4)$$

3.2.3. Text embedding

Traditional MTSC methods often overlook the rich semantic infor-

mation embedded within labels. To effectively bridge the modality gap and align time series signals with an LLM's semantic space, we introduce a structured text-embedding strategy. While LLMs possess vast world knowledge, direct unconstrained generation can introduce hallucination or instability. To address this, we leverage the LLM as a knowledge base under the guidance of expert supervision (Li et al., 2024). Specifically, we employ expert-designed prompts to query the LLM, ensuring it generates precise, physically meaningful descriptions for each class label (e.g., describing the kinematic characteristics of 'Walking'). Following the generation, the descriptions undergo a rigorous expert review process to verify their physical correctness. Consequently, we format our text input t_{text} as a structured string combining the label and its expert-guided description. This methodology enriches the simple tokens by providing explicit context and key features, establishing a stable and accurate semantic anchor for alignment. This structure is defined as:

Text = "label: " + $\langle \text{text_label} \rangle$ + ", description: " + $\langle \text{description} \rangle$ where $\langle \text{text_label} \rangle$ is the original dataset label (e.g., "Walking") and $\langle \text{description} \rangle$ is the text generated by the LLM using expert prompts. For example, in HAR datasets:

- 1) Text = "label: Walking, description: Normal movement on a flat surface with a rhythmic pattern."
- 2) Text = "label: Sitting, description: Relatively stationary position with minimal movement."

Following the definition of t_{text} , we select the BERT-base model (Devlin et al., 2019) as our text embedding backbone, balancing computational efficiency with representation capability. When this structured text sequence t_{text} is input, the BERT-base model converts each token within the sequence into a 768-dimensional embedding vector via its token embedding layer $E(\cdot)$, as shown in Eq. (5). These embedding vectors $x_{text} \in \mathbb{R}^{L_{text} \times D}$, where L_{text} denotes the sequence length of tokens, encapsulate both the semantic and contextual information of the tokens.

$$x_{text} = E(t_{text}) \quad (5)$$

3.3. Signal-Text alignment module

Given the local and global representations x_{local} , x_{global} of the time series and text label embeddings x_{text} , we propose a Signal-Text Alignment Module to thoroughly align these representations into a semantic space. This module addresses the alignment problem between the two modalities at two hierarchical levels: the token level and the prototype level. In the ME, the incorporation of learned queries with text embeddings introduces a degree of flexibility, effectively overcoming the constraints imposed by the exclusive use of static text labels. Subse-

quently, the cross-attention mechanism is employed to extract both explicit and implicit semantic elements within the time series, thereby achieving token-level alignment. Furthermore, drawing inspiration from contrastive learning principles, we align the textual representations from the CE with the multimodal outputs of the ME, fostering discriminative feature learning and parameter optimization through pairwise comparisons.

3.3.1. Construction of positive and negative signal-text pairs

The quality of signal-text pairs directly determines the effectiveness of cross-modal alignment. We define and construct pairs based on category consistency and semantic similarity, with detailed rules and processes as follows:

Positive pairs (S, T^+): Consist of a signal sample S and a text description T^+ belonging to the same category. A positive pair is formed by matching a signal sample with any text description corresponding to its ground-truth category label.

Negative pairs: Divided into two types to ensure sufficient supervision diversity. Ordinary negative pairs (S, T_1^-) consist of a time series sample S paired with a text description T_1^- from a different category, sampled randomly to cover all non-matching category combinations. Hard negative pairs (S, T_2^-) refer to a time series sample S paired with a text description T_2^- from a different category but with the highest semantic similarity, where similarity is measured by the cosine similarity between the features generated by the two encoder branches.

3.3.2. Token-level alignment

Matching Dataset Construction: To provide effective supervised signals for the matching loss in token-level alignment, we further construct a dedicated training dataset based on the pre-defined positive and negative signal-text pairs (Section 3.3.1). The dataset integrates three types of sample combinations: positive pairs (S, T^+), ordinary negative pairs (S, T_1^-) and hard negative pairs (S, T_2^-). Formally, a single training mini-batch D_{match} is expanded to size $3B$ and defined as a set of labeled tuples:

$$D_{match} = \{(S_k, T_k, Y_k) | k = 1, 2, \dots, 3B\} \quad (6)$$

where B denotes the input batch size, Y_k is a binary label indicating whether the pair (S_k, T_k) is positive or negative. It is defined as:

$$Y_k = \begin{cases} 1 & \text{if } (S_k, T_k) \text{ is a positive pair} \\ 0 & \text{if } (S_k, T_k) \text{ is a negative pair} \end{cases} \quad (7)$$

where S_k , T_k , and Y_k represent the signal sample, text description, and label in the batch, respectively.

Calculation Process: This method aims to achieve token-level alignment between time series and text labels through matching encoder trained on a newly constructed dataset. Cross-attention is a key technique for achieving fine-grained alignment. We integrate learnable queries $q \in \mathbb{R}^{L_q \times D}$, which are a set of randomly initialized parameters optimized during the alignment stage, with the text embedding $x_{text} \in \mathbb{R}^{L_{text} \times D}$ to construct the queries $Q_t \in \mathbb{R}^{(L_{text}+L_q) \times D}$. Here, the query length L_q is a hyperparameter set to 32, and $D = 768$ is consistent with the text representation space of the pre-trained model BERT-base. The introduction of learnable queries enables the extraction of latent semantic information from time series, rather than being confined to the given text labels. The inputs x_{global} and x_{local} are concatenated $x_{signal} = [x_{global}, x_{local}] \in \mathbb{R}^{L_{emb} \times D}$, where L_{emb} denotes the total length of the concatenated signal features. This signal representation is then projected via linear layers into keys K_s and values V_s . In order to reduce the gap between the two different modalities, we use the Q_t obtained from the text modality and learnable queries as joint queries to extract relevant information from the time series signal modality. Then, we further introduce shared layers $F_{shared}(\cdot)$ to map the fusion representations into a specific semantic space. The specific calculation process is as follows:

$$Q_t = SA_{mat}([q, x_{text}]) \quad (8)$$

$$H_{fusion} = \text{Softmax}\left(\frac{Q_t K_s^T}{\sqrt{D}}\right) V_s \quad (9)$$

$$z^{mat} = \text{ME}(x_{signal}, [q, x_{text}]) \\ = F_{shared}(H_{fusion}) \quad (10)$$

where $H_{fusion} \in \mathbb{R}^{(L_{text}+L_q) \times D}$ holds the same dimension as the dimension of queries. The final signal output is $z^{mat} \in \mathbb{R}^{(L_{text}+L_q) \times D}$.

Inspired by the CLIP (Radford et al., 2021), we define a fine-grained alignment objective function $\mathcal{L}_{mat}$ shown in Eq. (12). It is essentially a binary classification task, to determine whether the signal-text pair is a positive match or a negative match. We feed each final output z^{mat} into a matching head $F_{mat}(\cdot)$, which is a one-layer MLP.

$$p^{mat} = \text{Softmax}(F_{mat}(z^{mat})) \quad (11)$$

$$\mathcal{L}_{mat} = \mathbb{E}_{(X) \sim D_{match}} H(y^{mat}, p^{mat}) \\ = \mathbb{E}_{(X) \sim D_{match}} \left[- \sum y_i^{mat} \log(p_i^{mat}) \right] \quad (12)$$

where $H(\cdot, \cdot)$ denotes the cross-entropy function, y^{mat} denotes the true label, while p^{mat} represents the model prediction.

3.3.3. Prototype-level alignment

Beyond cross-attention focusing on local inter-modality interactions, we further leverage contrastive learning to bridge the distance between signal and text representation spaces. Unlike traditional contrastive learning relying on data augmentation, we extend beyond single modality constraints. Instead, we treat the processed time series signal and its corresponding text description as a positive pair, while text descriptions of other labels serve as negative pairs. Our CE processes a mixed sequence of text embeddings $x_{text'} \in \mathbb{R}^{(L_{text}+1) \times D}$, where the first token $x_{text'}^{(1)} = [\text{CLS}]$ is a special token, and generates a high-level semantic representation space $z_{text'} \in \mathbb{R}^{(L_{text}+1) \times D}$. To enhance the distinctiveness of text descriptions corresponding to different labels and to further widen the separation between them, we propose the incorporation of a classifier as the optimization objective $\mathcal{L}_{cls}$, shown in Eq. (15). When given the output $z_{text'}$, we construct a classification head $F_{cls}(\cdot)$, which is a one-layer MLP.

$$z_{text'} = \text{CE}(x_{text'}) \\ = F_{shared}(SA_{cls}(x_{text'})) \quad (13)$$

$$p^{cls} = \text{Softmax}(F_{cls}(z_{text'}^{(1)})) \quad (14)$$

$$\mathcal{L}_{cls} = \mathbb{E}_{(X) \sim D_{train}} H(y^{cls}, p^{cls}) \\ = -\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^M y_{ij}^{cls} \log(p_{ij}^{cls}) \quad (15)$$

where $z_{text'}^{(1)} \in \mathbb{R}^D$ is the first token which contains global information, y^{cls} is the ground truth label and p^{cls} is the prediction probability, M is the number of classes, and N is the number of training samples.

To address the challenge of cross-modal alignment between time series signals and text labels, particularly when multiple heterogeneous signals share identical text labels in a batch, a scenario that induces gradient ambiguity and exacerbates feature collapse, we propose a contrastive framework integrated with a dynamic masking technique. Our strategy first normalizes the output of z_{signal} and $z_{text'}^{(1)}$ to ensure consistent comparison.

$$S_i = \frac{\text{mean}(z_{\text{signal}})}{\|\text{mean}(z_{\text{signal}})\|_2} \quad (16)$$

$$= \frac{\text{mean}(\text{ME}(x_{\text{signal}}, q))}{\|\text{mean}(\text{ME}(x_{\text{signal}}, q))\|_2}$$

$$T_i = \frac{z_{\text{text}}^{(1)}}{\|z_{\text{text}}^{(1)}\|_2} \quad (17)$$

In Eq. (16) and Eq. (17), $S_i \in \mathbb{R}^D$ represents the signal feature for the i th sample, and $T_i \in \mathbb{R}^D$ is the corresponding text feature.

Then, we compute the cosine similarity between each signal feature and text feature, as defined in Eq. (18). To handle redundant matches, we introduce a binary mask M_{ij} that selectively focuses on unique signal-text pairs from raw dataset D_{raw} , defined as Eq. (19).

$$\text{sim}(i, j) = S_i T_j^T \quad (18)$$

$$M_{ij} = \begin{cases} 1, & \text{if } S_i \text{ and } T_j \text{ form a unique pair} \\ 0, & \text{otherwise} \end{cases} \quad (19)$$

We define the positive set $\mathcal{P}_i = \{j | \text{text}_j = \text{text}_i \text{ and } M_{ij} = 1\}$ and the negative set $\mathcal{N}_i = \{j | \text{text}_j \neq \text{text}_i\}$. The contrastive loss is computed as follows:

$$\mathcal{L}_{\text{con}} = -\frac{1}{B} \sum_{i=1}^B \log \frac{\sum_{j \in \mathcal{P}_i} M_{ij} \exp(\text{sim}(i, j)/\tau)}{\sum_{j \in \mathcal{P}_i} M_{ij} \exp(\text{sim}(i, j)/\tau) + \sum_{k \in \mathcal{N}_i} \exp(\text{sim}(i, k)/\tau)} \quad (20)$$

where τ is the temperature parameter.

Our strategy effectively employs the masking technique to mitigate redundant matching within batches, thereby significantly enhancing the robustness and accuracy of the alignment process. Specifically, by maximizing the similarity of positive pairs while minimizing the similarity of negative pairs, our model learns a discriminative representation space that captures the semantic correspondences between signals and text.

3.4. Training

The detailed training algorithm is depicted in Algorithm 1. Following the input of multivariate time series and text labels, along with the model initialization, the training proceeds through multiple iterations. In each iteration, time series and text features are extracted, followed by sequential execution of hybrid alignment, computation of the total loss $\mathcal{L}_{\text{total}}$, and parameter updates via gradient descent to complete the alignment training. Upon completion, the trained feature representation module and signal-text alignment module are frozen to serve as the foundation for the subsequent fine-tuning stage.

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{mat}} + \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{con}} \quad (21)$$

3.5. Task-aware tuning

In the field of time series analysis, cross-subject recognition, which aims to classify time series signals collected from different individuals, faces critical challenges from domain shift. Following the signal-text alignment stage, we address this downstream task by employing a parameter-efficient fine-tuning strategy designed to mitigate domain shift and enhance generalization. As illustrated in Fig. 3, this architecture harnesses the extensive prior knowledge of a frozen pre-trained LLM, introducing a trainable Adapter and a final Classification head. The adapter is designed to map the aligned time series representation z^{con} obtained from the alignment module into the semantic space that the pre-trained LLM can comprehend.

$$z_{\text{LLM}} = \text{Adapter}(z^{\text{con}}) \quad (22)$$

Algorithm 1

Training process for alignment stage.

-
- 1: **Input:** Raw time series $x \in \mathbb{R}^{B \times L \times D}$ with samples B , length L , and variables D . Text labels $t_{\text{text}} \in \mathbb{R}^{B \times L_{\text{text}}}$, where L_{text} represents the sequence length. The number of epochs is N .
 - 2: **Initialize:** Global Encoder $G(\cdot)$, Local Encoder $L(\cdot)$, Text Embedding $E(\cdot)$, Classification Encoder $\text{CE}(\cdot)$, and Matching Encoder $\text{ME}(\cdot)$, learnable queries $q \in \mathbb{R}^{L_q \times D}$.
 - 3: **For** $i \in [1, \dots, N]$ **do**
 - 4: **Feature Representation**
 - 5: $x_{\text{global}} = G(x)$ #Get global features
 - 6: $x_{\text{local}} = L(x)$ #Get local features
 - 7: $x_{\text{signal}} = [x_{\text{global}}, x_{\text{local}}]$ #Feature concatenation
 - 8: $x_{\text{text}} = E(t_{\text{text}})$ #Text embedding
 - 9: **Alignment Stage**
 - 10: **Token level alignment**
 - 11: If the signal-text pairs dataset does not exist
 - 12: $\hat{z}_{\text{text}} = \text{CE}(x_{\text{text}})$
 - 13: $\hat{z}_{\text{signal}} = \text{ME}(x_{\text{signal}}, [q, x_{\text{text}}])$
 - 14: $\text{Matrix} = \cos(\hat{z}_{\text{text}}, \hat{z}_{\text{signal}}) = \frac{\hat{z}_{\text{text}} \cdot \hat{z}_{\text{signal}}^T}{\|\hat{z}_{\text{text}}\| \|\hat{z}_{\text{signal}}\|}$
 - 15: Based on the *Matrix*, constructing a new training set (X, Y) , as shown in Eq. (6-7).
 - 16: Return 4. #Get new x_{signal} and x_{text} .
 - 17: $z^{\text{mat}} = \text{ME}(x_{\text{signal}}, [q, x_{\text{text}}])$
 - 18: $p^{\text{mat}} = \text{Softmax}(\text{F}_{\text{mat}}(z^{\text{mat}}))$
 - 19: $\mathcal{L}_{\text{mat}} = \mathbb{E}_{(x) \sim D_{\text{match}}} H(y^{\text{mat}}, p^{\text{mat}})$ #Calculate matching loss.
 - 20: **Prototype level alignment**
 - 21: $z_{\text{text}} = \text{CE}(x_{\text{text}})$
 - 22: $p^{\text{cls}} = \text{Softmax}(\text{F}_{\text{cls}}(z_{\text{text}}^{(1)}))$
 - 23: $\mathcal{L}_{\text{cls}} = \mathbb{E}_{(x_{\text{text}}) \sim D_{\text{new}}} H(y^{\text{cls}}, p^{\text{cls}})$ #Calculate the classification loss.
 - 24: $z_{\text{signal}} = \text{ME}(x_{\text{signal}}, q)$
 - 25: $\mathcal{L}_{\text{con}} = \text{Contrastive}(z_{\text{text}}^{(1)}, z_{\text{signal}})$ #Calculate contrastive loss.
 - 25: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{mat}} + \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{con}}$ #Calculate total loss.
 - 26: Update with $\nabla \mathcal{L}_{\text{total}}$ #Optimize model parameters.
 - 27: **End for**
-

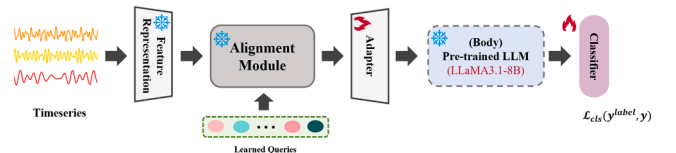


Fig. 3. LLM fine-tuning architecture with trainable adapter and classification head, harnessing the prior knowledge of a large language model (LLM). Freezing most modules (Feature representation, Alignment Module and Pre-trained LLM) preserves generalizability, while training the adapter and classifier.

The features, after processing by the adapter, are fed into the main body of the LLM, which employs the LLaMA3.1-8B model in this study. Within the LLM body, the features undergo further extraction and processing, thereby yielding a more profound semantic representation. Subsequently, these processed features are input into the classification head $\text{F}_{\text{cls}}(\cdot)$ to generate the final prediction, as defined in Eq. (23).

$$p = \text{F}_{\text{cls}}(\text{LLM}(z_{\text{LLM}})) \quad (23)$$

During the fine-tuning phase, our objective is to minimize the cross-entropy loss between the predicted and true labels, which is formally expressed as:

$$\mathcal{L}_{\text{pre}} = - \sum_{i=1}^C y_i \log(p_i) \quad (24)$$

where Cl denotes the total number of classes, y is the one-hot encoding of the true label, and pre represents the probability distribution of the predicted label.

4. Experiments

4.1. Experimental settings

This experiment consists of two stages: Alignment-stage pre-training and Classification-stage fine-tuning. All code was developed using the PyTorch framework and executed on an NVIDIA GeForce RTX 4090 GPU. For semantic extraction of text labels, we utilized the word embeddings of the pre-trained BERT-Base model, whose parameters were subsequently frozen during the training process. Subsequently, we trained the time series encoder module and the alignment module from scratch to achieve alignment between signals and text labels. Finally, the processed signals were fed into the LLaMA3.1-8B model to complete the downstream classification task.

Alignment-stage Configuration: We employed the AdamW optimizer with an initial learning rate of $5e-5$ and used the Cosine Annealing with warm restarts strategy. The learning rate was restarted every 500 training iterations, with a minimum learning rate set to $1e-8$. To enhance training efficiency and stability, we adopted mixed-precision training with gradient scaling, where the numerical precision of the backbone network was set to FP16. For signal-text alignment, we constructed three vanilla Transformer blocks. The length of the query token was set to 32. Beyond the standard self-attention in each block, we strategically integrated a cross-attention layer between every two consecutive Transformer blocks to facilitate interaction.

Classification-stage Configuration: The same set of hyperparameters used in the pre-training stage was retained, except for the initial learning rate of $1.5e-4$. We froze all model modules except the Adapter and final classification head, which were trained. Additionally, the parameters of the frozen LLM were converted to the half-precision floating-point format (FP16). Due to these measures, our model maintained predictive accuracy while significantly enhancing computational efficiency.

4.1.1. Dataset

In this study, we validated the proposed ST-Align on four benchmark datasets spanning distinct domains (Table 2). Given the disparities among the datasets, we performed truncation or padding on each dataset and normalized the data to ensure uniformity. To comprehensively evaluate the performance of the proposed ST-Align, the selected datasets encompass air writing recognition, human activity recognition, and sleep state monitoring, which involve entirely distinct types of signals. Moreover, the variations in sample size and dimensionality among these datasets pose significant challenges to the model's generalization capability. The details are as follows:

HAR²: The dataset is a comprehensive collection of human activity recognition data, gathered from the IMUs in smartphones. It comprises 10,299 samples, each with nine dimensions corresponding to the sensor channels. These samples capture the movement patterns of 30 participants as they engage in 6 distinct daily activities: standing, sitting,

walking, walking downstairs, walking upstairs, and lying down.

Air Writing (AW-A)³: The dataset encompasses 21,450 six-dimensional samples, capturing the gestures of writing 26 English uppercase letters. These samples are derived from acceleration and gyroscope data, collected at a sampling rate of 62 Hz using a Microsoft Band worn on the dominant hand. The dataset features contributions from 55 participants, providing a rich and diverse collection of air writing gestures for research and development purposes.

WISDM⁴: This is a comprehensive human activity recognition dataset that captures 3-axis accelerometer measurements from smartphones worn by participants. Specifically, it includes data collected from 35 participants at a sampling rate of 20 Hz. We employ non-overlapping segments consisting of 128-time-step. The dataset features six distinct activity labels.

Sleep-EDF(S-EDF)⁵: The dataset is a publicly available resource for sleep stage classification, containing 20,626 EEG recordings from 20 healthy individuals. The dataset is designed to classify EEG signals into six distinct sleep stages: wakefulness (W), non-rapid eye movement stages (N1, N2, N3), rapid eye movement (REM), and an unknown state (Unknown).

4.1.2. Baselines and evaluation

To comprehensively evaluate the performance of our ST-Align (w/ and w/o LLM) and demonstrate its superiority over state-of-the-art (SOTA) approaches, we compare our method against 12 baselines spanning three distinct categories. First, to benchmark against efficient non-deep learning algorithms, we implemented the ROCKET family methods, including MiniRocket (Dempster et al., 2021) and MultiRocket (Tan et al., 2022). Second, we selected representative deep learning architectures to evaluate feature representation capabilities, comprising ConvTran (Foumani et al., 2024), Reformer (Kitaev et al., 2020), Informer (Zhou et al., 2021), Nonstationary Transformer (Liu et al., 2023b), PatchTST (Nie et al., 2023), TimsNet (Wu et al., 2023) and iTransformer (Liu et al., 2024a). Third, to benchmark against the emerging LLM4TS landscape, we incorporated Time-LLM (Jin et al., 2024), GPT4TS (T. T. Zhou et al., 2023), and TEST (Sun et al., 2024) as representative foundation model baselines. The evaluation metric is the mean accuracy calculated on the test datasets, and each experiment was repeated at least five times with different random seeds to ensure statistical reliability.

4.2. Overall comparison with MTSC benchmarks

Unlike the conventional setting including data from all subjects in the training dataset, we focus exclusively on cross-subject prediction results. We employ a 10-fold cross-subject validation (10f-CS), where the entire dataset is partitioned into ten subsets based on subject IDs, as detailed in Table 3. To comprehensively evaluate the models' generalization capability and stability, we present the statistical distribution of accuracy for each model via a box plot, as illustrated in Fig. 4. This visualization highlights not only the median performance but also the variance across different folds. The cross-subject experiment is particularly valuable for assessing the model's capacity to predict results for unseen subjects.

As shown in Table 3, ST-Align achieves a notable improvement in recognition accuracy across all four datasets compared to the state-of-the-art (SOTA) models, with an accuracy rate reaching 88.4%. Even without the pre-trained LLM, ST-Align (w/o LLM) still demonstrates favorable performance, with an average recognition accuracy of 86.3%,

Table 2

The key features of the 4 benchmark datasets.

Datasets	Subjects	Length	Channels	Classes	Samples
HAR	30	128	9	6	10,299
AW	55	155	6	26	21,450
WISDM	35	120	3	6	8575
S-EDF	20	3000	1	6	20,626

² <https://github.com/aannasw/uci-har/tree/master/data/UCI-HAR-Dataset>.

³ http://bigdatalab.tau.ac.il/shared_resources/datasets/airwriting_dataset.zip.

⁴ <https://www.cis.fordham.edu/wisdm/dataset.php>.

⁵ <https://archive.physionet.org/physiobank/database/sleep-edfx/sleep-cassette/>.

Table 3

ST-Align results of 10-fold Cross-Subject (10f-CS) evaluations across 4 benchmark datasets. Average represents the average accuracy achieved by each method across 4 datasets. Improve represents the percentage performance of our model compared to the baseline model. The best results are bolded and second-best results are underlined in all following tables.

10f-CS	ROCKET			Deep Learning Architectures							LLM4TS		Ours	
	Minirocket	Multirocket	ConvTran	Reformer	Informer	Nonstationary Transformer	PatchTST	TimesNet	iTransformer	TimeLLM	GPT4TS	TEST	ST-Align (w/o LLM)	ST-Align
HAR 1	0.930	<u>0.919</u>	0.873	0.887	0.880	0.880	0.870	0.892	0.903	0.807	0.844	0.861	0.841	0.918
HAR 2	0.925	<u>0.911</u>	0.837	0.842	0.856	0.831	0.868	0.831	0.895	0.767	0.777	0.882	0.895	<u>0.911</u>
HAR 3	0.943	<u>0.948</u>	0.892	0.902	0.920	0.916	0.881	0.871	0.937	0.784	0.896	0.921	0.913	0.959
HAR 4	<u>0.951</u>	0.936	0.931	0.950	0.909	0.951	0.853	0.901	0.952	0.760	0.943	0.897	0.936	0.966
HAR 5	0.959	0.899	0.825	0.866	0.879	0.905	0.767	0.901	0.882	0.716	0.801	0.869	0.884	<u>0.915</u>
HAR 6	0.956	0.942	0.881	<u>0.948</u>	0.940	0.907	0.852	0.930	0.928	0.742	0.918	0.898	0.878	0.924
HAR 7	<u>0.988</u>	0.983	0.942	0.940	0.981	0.985	0.902	0.917	0.983	0.769	0.949	0.981	0.977	0.996
HAR 8	<u>0.968</u>	0.979	0.953	<u>0.988</u>	0.998	0.986	0.868	0.945	0.987	0.788	0.962	0.954	0.969	0.986
HAR 9	<u>0.990</u>	0.953	0.957	0.949	0.949	0.972	0.848	0.969	0.975	0.822	0.970	0.941	0.965	0.993
HAR 10	0.948	0.933	0.946	0.923	0.931	<u>0.949</u>	0.839	0.924	0.975	0.769	0.956	0.899	0.927	0.945
HAR Avg	0.965±0.021	0.940±0.026	0.904±0.046	0.920±0.042	0.924±0.043	<u>0.928±0.047</u>	0.855±0.034	0.908±0.037	0.942±0.037	0.772±0.029	0.902±0.067	0.910±0.037	0.919±0.042	<u>0.951±0.032</u>
AW-A 1	0.667	0.654	0.597	0.720	0.715	0.748	0.647	0.737	0.721	0.598	0.667	0.801	<u>0.852</u>	0.861
AW-A 2	0.736	0.698	0.706	0.754	0.769	0.773	0.703	0.760	0.785	0.623	0.720	0.786	<u>0.829</u>	0.871
AW-A 3	0.653	0.651	0.633	0.730	0.756	0.771	0.662	0.725	0.701	0.593	0.679	0.764	<u>0.832</u>	0.848
AW-A 4	0.797	0.801	0.746	0.792	0.834	0.834	0.783	0.814	0.802	0.644	0.771	0.852	<u>0.877</u>	0.889
AW-A 5	0.821	0.813	0.776	0.834	0.849	0.869	0.794	0.826	0.809	0.641	0.763	0.835	<u>0.899</u>	0.912
AW-A 6	0.776	0.757	0.747	0.819	0.819	0.853	0.753	0.840	0.788	0.686	0.744	0.884	<u>0.893</u>	0.894
AW-A 7	0.611	0.612	0.601	0.663	0.711	0.723	0.619	0.696	0.696	0.585	0.621	0.721	<u>0.776</u>	0.786
AW-A 8	0.625	0.605	0.607	0.693	0.700	0.732	0.648	0.728	0.671	0.605	0.560	0.746	<u>0.767</u>	0.806
AW-A 9	0.733	0.711	0.698	0.792	0.781	0.827	0.728	0.796	0.744	0.620	0.671	0.854	<u>0.873</u>	0.895
AW-A 10	0.723	0.726	0.568	0.687	0.696	0.753	0.667	0.717	0.679	0.589	0.617	0.697	<u>0.819</u>	0.832
AW-A Avg	0.714±0.069	0.703±0.069	0.668±0.071	0.755±0.056	0.763±0.054	0.795±0.049	0.700±0.058	0.764±0.046	0.740±0.050	0.618±0.030	0.681±0.066	0.794±0.059	<u>0.842±0.044</u>	0.859±0.039
WISDM 1	0.915	0.903	0.808	0.881	0.872	0.884	0.794	0.891	0.885	0.727	0.868	0.897	<u>0.935</u>	0.944
WISDM 2	0.861	0.856	0.676	0.801	0.700	0.826	0.841	<u>0.863</u>	0.788	0.843	0.710	0.857	0.859	0.865
WISDM 3	<u>0.927</u>	0.905	0.814	0.881	0.827	0.895	0.888	0.878	0.822	0.786	0.815	0.901	0.919	0.935
WISDM 4	0.917	0.899	0.823	0.884	0.856	0.892	0.906	0.869	0.853	0.799	0.823	0.848	<u>0.928</u>	0.956
WISDM 5	0.843	0.842	0.799	0.738	0.743	0.835	0.780	0.845	0.853	0.766	0.839	0.818	<u>0.868</u>	0.893
WISDM 6	0.774	0.777	0.754	0.719	0.817	<u>0.905</u>	0.810	0.840	0.756	0.719	0.762	0.894	0.903	0.919
WISDM 7	<u>0.892</u>	0.884	0.792	0.853	0.849	0.856	0.867	0.850	0.864	0.765	0.789	0.897	0.880	0.889
WISDM 8	0.914	0.888	0.770	0.862	0.867	0.871	0.819	0.842	0.810	0.654	0.745	0.886	<u>0.893</u>	0.914
WISDM 9	0.910	0.903	0.823	0.886	0.912	0.936	0.885	0.813	0.823	0.819	0.824	0.895	0.916	<u>0.931</u>
WISDM 10	0.923	0.911	0.817	0.907	0.855	0.861	0.758	0.901	0.802	0.797	0.755	0.846	<u>0.925</u>	0.961
WISDM Avg	0.888±0.046	0.877±0.040	0.788±0.043	0.841±0.063	0.830±0.060	0.876±0.032	0.835±0.048	0.860±0.025	0.826±0.037	0.767±0.052	0.793±0.047	0.874±0.028	<u>0.903±0.025</u>	0.921±0.029
S-EDF 1	0.639	0.635	0.637	0.675	0.741	0.702	0.587	0.773	0.645	0.530	0.588	0.779	<u>0.806</u>	0.825
S-EDF 2	0.782	0.788	0.710	0.793	0.789	0.792	0.666	0.800	0.744	0.639	0.616	0.796	<u>0.836</u>	0.843
S-EDF 3	0.656	0.615	0.706	0.788	<u>0.807</u>	0.801	0.661	0.801	0.689	0.472	0.637	0.804	0.792	0.819
S-EDF 4	0.561	0.579	0.557	0.696	0.640	0.578	0.454	0.625	0.585	0.506	0.602	0.627	0.658	<u>0.693</u>
S-EDF 5	0.722	0.695	0.634	0.718	0.738	0.713	0.597	0.738	0.681	0.521	0.556	0.713	<u>0.746</u>	0.755
S-EDF 6	0.792	0.754	0.732	0.759	0.781	0.785	0.644	0.787	0.732	0.692	0.621	0.699	<u>0.799</u>	0.815
S-EDF 7	0.674	0.675	0.703	0.780	<u>0.781</u>	0.797	0.617	0.768	0.697	0.556	0.656	0.741	0.773	0.792
S-EDF 8	0.692	0.689	0.667	0.747	0.804	0.762	0.586	0.741	0.698	0.687	0.671	0.800	<u>0.815</u>	0.826
S-EDF 9	0.732	0.731	0.710	0.798	<u>0.801</u>	0.777	0.626	0.797	0.715	0.625	0.580	0.799	<u>0.801</u>	0.813
S-EDF 10	0.692	0.688	0.765	0.828	<u>0.832</u>	0.829	0.684	0.826	0.774	0.682	0.652	0.871	0.834	<u>0.846</u>
S-EDF Avg	0.694±0.065	0.685±0.060	0.682±0.057	0.758±0.045	0.770±0.054	0.754±0.069	0.612±0.062	0.766±0.054	0.696±0.050	0.591±0.079	0.618±0.035	0.763±0.066	<u>0.786±0.050</u>	0.803±0.044
Average	0.815	0.801	0.761	0.819	0.822	0.838	0.751	0.845	0.801	0.687	0.749	0.835	<u>0.863</u>	0.884
Improve	8.47%	10.36%	16.16%	7.90%	7.50%	5.40%	17.7%	4.60%	10.3%	28.68%	18.02%	5.87%	2.40%	-

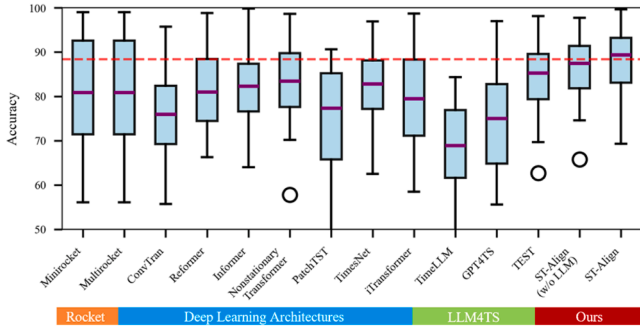


Fig. 4. Box plot illustrating the performance distribution and stability comparison across different model categories.

achieving competitive level. The incorporation of the LLM enhances the recognition accuracy and reduces the variability by 10.6%, thereby improving the model's generalization capability across different subjects and mitigating overfitting to the training dataset.

To thoroughly validate the effectiveness of our approach, we break down the comparison across three specific baseline categories:

Comparison with Efficient Non-Deep Learning Methods: While methods like MiniRocket exhibit strong performance on kinematic-intensive datasets like HAR and WISDM (e.g., MiniRocket achieves 96.5% on HAR), they struggle with complex distribution shifts, dropping to 71.4% on AW-A. In sharp contrast, ST-Align maintains robust performance on AW-A with 85.9% accuracy. This disparity stems from the Rocket family's reliance on statistical features generated by random kernels, which function effectively for distinct physical patterns but lack the semantic adaptability required for complex cross-subject generalization.

Comparison to Deep Learning Architectures: ST-Align consistently surpasses advanced deep learning models, which often leads to overfitting subject-specific patterns and failing to generalize against domain shifts. In contrast, ST-Align overcomes this by introducing semantic guidance. By aligning diverse temporal signals with stable textual prototypes, our method forces the model to learn features that are semantically meaningful, ensuring robustness across subjects.

Comparison with LLM4TS benchmarks: ST-Align significantly outperforms Time-LLM (68.7% average accuracy). This performance gap is likely because Time-LLM is primarily structured for forecasting via reprogramming, leading to suboptimal adaptation for classification tasks. Although GPT4TS and TEST are designed to handle both classification and forecasting, ST-Align still establishes superior performance. This confirms that our alignment strategy is more effective for classification tasks than other LLM baselines.

These comparative results indicate that ST-Align can effectively extract features from temporal signals and achieve robust performance across cross-subject datasets. Moreover, leveraging the pre-trained knowledge from the LLM further enhances the robustness to distribution shift. This highlights the potential of our framework for practical applications in real-world scenarios.

4.3. Ablation studies

In this section, we conduct ablation studies to evaluate the impact of key components in ST-Align, including time-series feature extraction, signal-text alignment, and the selection of query length. Comparative results highlight the significance of each component in enhancing the performance of time series classification. We selected the WISDM dataset as a representative benchmark due to its diverse activity classes and moderate scale. Furthermore, given the extensive configurations involved and the rigorous 10-fold cross-subject validation employed, conducting full ablation studies across all datasets would incur excessive computational overhead. Focusing on this representative dataset allows

for a detailed analysis of component contributions while maintaining computational feasibility. Subsequently, we will elaborate on the specific results of each ablation experiment.

4.3.1. Ablation results of the feature extraction

We compared various backbone networks in terms of performance for cross-subject recognition, with detailed results presented in Table 4. We initially employed two simple feature extraction networks: ResNet-10 and two Transformer blocks. The results indicated that while ResNet-10 accelerated model convergence on the training dataset, its generalization performance was poor; in contrast, the Transformer exhibited the opposite characteristics. We hypothesize that this disparity is attributed to the convolution mechanism, which struggles to capture long-range dependencies in continuous time series data. Based on this observation, we attempted to combine the ResNet-10 and Transformer modules, but the results remained unsatisfactory. Consequently, we further reduced the number of convolutional layers and adopted Patch Embedding as the backbone network, termed PatchConv, which yielded remarkable results. However, given that Patch Embedding is still fundamentally based on the convolutional mechanism, we augmented it with two additional Transformer layers to capture global time series dependencies, which corresponds to our proposed method.

4.3.2. Ablation results of the ST-Alignment module

We analyze the impact of the two-stage alignment method, Token-Level Alignment (TLA) and Prototype-Level Alignment (PLA) on model performance via ablation experiments on the WISDM dataset, with the focus on the role of different loss functions at each stage, as shown in Table 5. These studies isolate the effect of individual modules, offering insights into their specific roles and synergistic contributions to state-of-the-art performance.

Impact of TLA: When TLA is introduced ($\mathcal{L}_{mat}$), the average performance of the model increases significantly from 68.5% (baseline, $\mathcal{L}_{cls}$ only) to 81.0%. This stage bridges the gap between raw features and basic semantic units, laying a foundation for prototype alignment in the subsequent stage.

Impact of PLA: Building upon TLA ($\mathcal{L}_{mat}$), adding PLA ($\mathcal{L}_{con}$) further boosts average performance to 90.3%, representing improvement of 9.3% compared to the TLA-only stage. The loss $\mathcal{L}_{con}$ promotes the aggregation of features at the prototype level.

The combination of TLA and PLA achieves a far higher performance gain than either stage alone; the baseline $\rightarrow$ TLA $\rightarrow$ TLA+PLA progression shows a cumulative improvement of 21.8%. This confirms the necessity and complementarity of two-stage alignment. The result also shows that replacing the linear classifier with a pre-trained LLM further improves performance (from 90.3% to 92.1%), indicating the effectiveness of the alignment method and the time series classification capability of the LLM. However, the core performance boost still originates from the two-stage alignment framework, as TLA+PLA already achieves superior results compared to the linear classifier baseline. Importantly, the two-stage alignment framework demonstrates remarkable generalizability across diverse time series modalities.

4.3.3. Ablation results of query length

This ablation study, illustrated in Fig. 5, investigates the impact of query length on model performance, uncovering a non-linear correlation between query length and accuracy. Increasing the query length from 1 to 32 is associated with improved accuracy, indicating that a longer query length facilitates more thorough extraction of signal information, thereby enhancing model efficacy. Conversely, extending the query length to 48 correlates with a decrease in accuracy. This reduction is attributed to the incorporation of redundant information, which may overwhelm the model, leading to a dilution of pertinent temporal cues and consequently, a degradation in performance. The results highlight the significance of identifying an optimal query length to balance the richness of information with the model's processing capacity, thus

Table 4

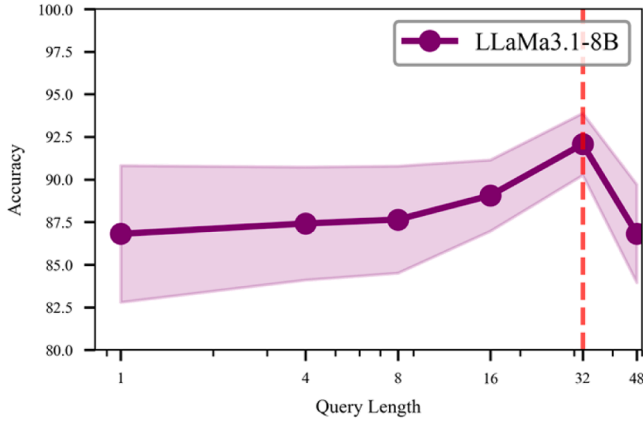
Ablation study on backbone architectures for multivariate time series feature extraction.

Backbone	1	2	3	4	5	6	7	8	9	10	Avg (10f-CS)
ResNet-10	0.753	0.627	0.616	0.734	0.748	0.616	0.720	0.746	0.753	0.627	0.702±0.055
Transformer	0.880	0.656	0.794	0.884	0.754	0.723	0.823	0.903	0.880	0.656	0.813±0.076
ResNet-10 + Transformer	0.856	0.681	0.825	0.813	0.804	0.692	0.847	0.891	0.856	0.681	0.812±0.068
PatchConv	<u>0.913</u>	<u>0.854</u>	<u>0.928</u>	<u>0.955</u>	<u>0.863</u>	<u>0.84</u>	0.905	<u>0.910</u>	<u>0.913</u>	<u>0.854</u>	<u>0.904±0.038</u>
PatchConv + Transformer	0.944	0.865	0.935	0.956	0.893	0.919	<u>0.889</u>	0.914	0.944	0.865	0.921±0.029

Table 5

Ablation study of a two-stage hybrid alignment method: dissecting token - level alignment (TLA) and prototype - level alignment (PLA) on WISDM dataset. Each row represents a unique ablation setup ('✓' indicates the corresponding module is included).

Vanilla	TLA	PLA	Linear	LLM	1	2	3	4	5	6	7	8	9	10	Avg(10f)
✓			✓		0.657	0.581	0.653	0.738	0.745	0.621	0.716	0.679	0.657	0.581	0.685±0.055
✓	✓		✓		0.724	<u>0.862</u>	0.843	0.848	0.748	0.711	0.812	0.799	0.724	0.856	0.810±0.060
✓	✓	✓	✓		<u>0.935</u>	0.859	<u>0.919</u>	<u>0.928</u>	<u>0.868</u>	<u>0.903</u>	<u>0.880</u>	<u>0.893</u>	<u>0.935</u>	<u>0.859</u>	<u>0.903±0.025</u>
✓	✓	✓		✓	0.944	0.865	0.935	0.956	0.893	0.919	0.889	0.914	0.944	0.865	0.921±0.029

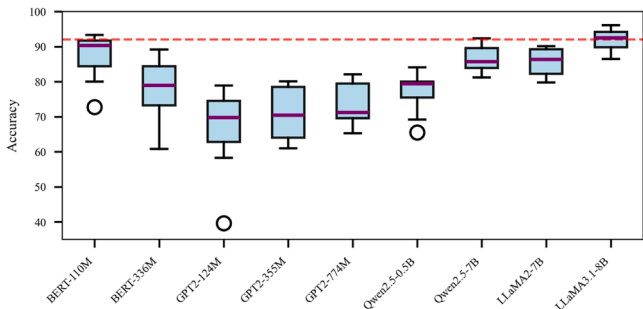
**Fig. 5.** Ablation study of query length.

optimizing the model's effectiveness in temporal sequence analysis.

4.4. Analysis of the pre-trained LLM

The pre-trained LLM plays a crucial role in determining the final performance of the model. To systematically evaluate the impact of pre-trained models at different scales, we compared several models, including BERT (110 M parameters, 12 layers, and 336 M parameters, 24 layers), GPT (124 M parameters, 12 layers, 335 M parameters, 24 layers, and 774 M parameters, 36 layers), Qwen2.5 (0.5B parameters, 24 layers, and 7B parameters, 28 layers), LLaMA2 (7B parameters, 32 layers), and LLaMA3.1 (8B parameters, 32 layers), as shown in Fig. 6.

Our findings indicate that the BERT series models exhibit remarkable

**Fig. 6.** Impact of pre-trained LLM scale on model classification accuracy.

performance even with relatively small model scales. For other series of LLMs, performance generally increases with model scale. Notably, the application of the LLaMA3.1 model directly enhances the prediction performance of ST-Align (w/o LLM) by 1.8%. This result suggests that the introduction of large models is not always effective and compatibility is key. Specifically, compatibility between the signal-text alignment method and downstream LLMs profoundly affects model performance (Yosinski et al., 2014). The alignment stage is pre-trained around the semantic space of the BERT-base text encoder. This design choice gives the BERT-110 M model a distinct compatibility advantage when used downstream, resulting in its strong performance. Conversely, while the BERT-336 M model shares the same architecture, its larger scale (24 layers) and higher dimensional feature space means it possesses a different and more complex semantic space than the 12-layer BERT-base. This creates a semantic mismatch that degrades performance, suggesting compatibility is highly specific to the exact embedding space used during alignment. This mismatch is even more significant for models with fundamentally different architectures, such as the GPT or Qwen series, which suffer from poor performance. In contrast, very large-scale models, like LLaMA3.1-8B, appear to have sufficient generalization capabilities to overcome this initial incompatibility (Kaplan et al., 2020; Wei et al., 2022). Overall, these findings underscore the importance of LLM scale and alignment method compatibility to achieve optimal cross-modal representation learning in time series classification tasks.

4.5. Sensitivity and training stability analysis

The temperature coefficient, as a key regulatory hyperparameter, demonstrates a unique mechanism of action. As shown in Fig. 7(a), as the temperature coefficient τ increases from 0.01 to 0.4, the average model performance exhibits a significant inverted-U trend. When the temperature coefficient τ is set to 0.07, the model achieves optimal accuracy and generalizability. This inverted-U relationship underscores the critical role of temperature coefficient tuning in balancing feature similarity and discriminability, providing a theoretical basis for optimizing cross-modal alignment in time series classification.

To evaluate training stability, we monitor loss dynamics during training and present the visualization of loss curves in Fig. 7(b). The training process exhibits remarkable stability, as evidenced by the consistent convergence of all loss components. These stable training dynamics highlight that our designed loss functions effectively complement each other, fostering robust feature interactions.

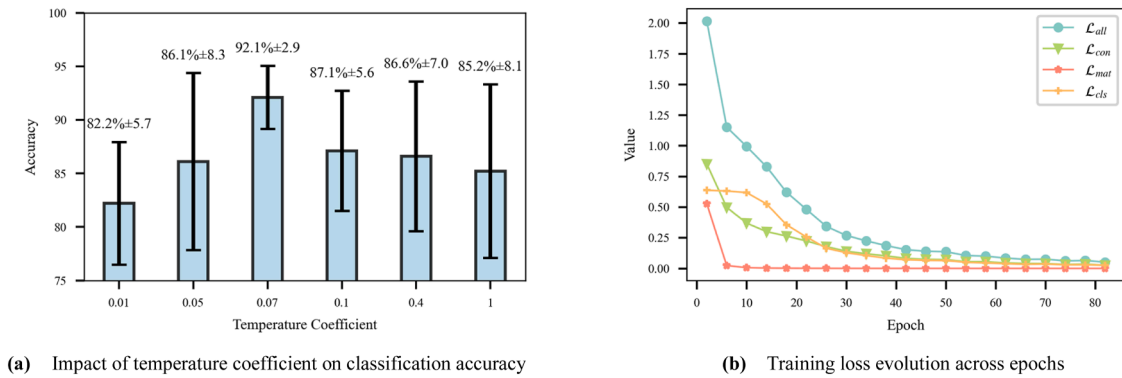


Fig. 7. Sensitivity analysis: this figure presents two complementary analyses to characterize model sensitivity, spanning hyperparameter tuning and training convergence.

5. Conclusion

In this paper, we propose ST-Align, a novel framework that aims to improve time series classification by integrating LLM. Unlike existing methods, which typically focus on individual alignment components, ST-Align introduces an innovative hybrid alignment strategy: token-level alignment and prototype-level alignment. The key advantages of ST-Align are as follows:

- By designating text labels as anchors, temporal signals are fused with intrinsic semantics and mapped to a semantic space, eliminating the need for supplementary textual information and reducing data preparation effort.
- ST-Align introduces an innovative two-stage "micro-macro" hybrid alignment paradigm by integrating fine-grained token alignment with semantic prototype clustering, enhancing the consistency between representations and semantic space.
- The modular independence facilitates the seamless integration of LLMs, offering versatility that extends beyond the specific large models discussed herein. For new model deployment, fine-tuning only the Adapter module suffices to significantly reduce training costs. Although incorporating the LLM incurs higher computational overhead during inference, it enhances the model's generalization capability in complex cross-subject scenarios while maintaining a highly parameter-efficient training design. This offers a versatile trade-off for deployment.

In the future, we intend to investigate the potential of larger LLMs in enhancing the accuracy of time series classification, to address the demands of real-world scenarios. We hope that the proposed signal-text alignment paradigm will contribute to the MTSC research community.

Data availability

Data will be made available on request.

CRediT authorship contribution statement

Zhenghuang Wu: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Tao Zhang:** Writing – review & editing, Writing – original draft, Software, Methodology, Data curation. **Ke Li:** Writing – review & editing, Supervision, Resources, Project administration, Funding acquisition. **Yuangan Li:** Writing – review & editing, Project administration.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the Chinese National Natural Science Foundation (No. 61773039), and Aeronautical Science Foundation of China (No. 2024Z031051001)

References

- Chen, T., Kornblith, S., Norouzi, M., Hinton, G., 2020. A simple framework for contrastive learning of visual representations. <https://doi.org/10.48550/arXiv.2002.05709>.
- Cheng, M., Chen, Y., Liu, Q., Liu, Z., Luo, Y., 2024. Advancing time series classification with multimodal language modeling. <https://doi.org/10.48550/arXiv.2403.12371>.
- Dempster, A., Schmidt, D. F., & Webb, G. I. (2021). MINIROCKET: A very fast (Almost) deterministic transform for time series classification. In *Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining* (pp. 248–257). <https://doi.org/10.1145/3447548.3467231>
- Devlin, J., Chang, M.-W., Lee, K., Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. <https://doi.org/10.48550/arXiv.1810.04805>.
- Eldele, E., Ragab, M., Chen, Z., Wu, M., Kwok, C.K., Li, X., Guan, C., 2021. Time-series representation learning via temporal and contextual contrasting. <https://doi.org/10.48550/arXiv.2106.14112>.
- Foumani, N. M., Tan, C. W., Webb, G. I., & Salehi, M. (2024). Improving position encoding of transformers for multivariate time series classification. *Data Mining and Knowledge Discovery*, 38, 22–48. <https://doi.org/10.1007/s10618-023-00948-2>
- Garza, A., Challu, C., Mergenthaler-Canseco, M., 2024. TimeGPT-1. <https://doi.org/10.48550/arXiv.2310.03589>.
- Guo, R., Wei, J., Sun, L., Yu, B., Chang, G., Liu, D., Zhang, S., Yao, Z., Xu, M., Bu, L., 2024. A survey on image-text multimodal models. <https://doi.org/10.48550/arXiv.2403.09157>.
- Han, W., Qiu, J., Zhu, J., Xu, M., Weber, D., Li, B., Zhao, D., 2024. Can brain signals reveal inner alignment with Human languages? <https://doi.org/10.48550/arXiv.2208.06348>.
- He, K., Fan, H., Wu, Y., Xie, S., Girshick, R., 2020. Momentum contrast for unsupervised visual representation learning.
- He, K., Zhang, X., Ren, S., Sun, J., 2015. Deep residual learning for image recognition. <https://doi.org/10.48550/arXiv.1512.03385>.
- Hu, Y., Li, Q., Zhang, D., Yan, J., Chen, Y., 2025. Context-alignment: Activating and enhancing LLM capabilities in time series. <https://doi.org/10.48550/arXiv.2501.03747>.
- Huang, H., Chen, J., Sun, R., & Wang, S. (2022). Short-term traffic prediction based on time series decomposition. *Physica A: Statistical Mechanics and its Applications*, 585, Article 126441. <https://doi.org/10.1016/j.physa.2021.126441>
- Jiang, Y., Pan, Z., Zhang, X., Garg, S., Schneider, A., Nevmyvaka, Y., Song, D., 2024. Empowering time series analysis with large language models: A survey. <https://doi.org/10.48550/arXiv.2402.03182>.
- Kaplan, J., McCandlish, S., Henighan, T., Brown, T.B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., Amodei, D., 2020. Scaling laws for neural language models. <https://doi.org/10.48550/arXiv.2001.08361>.
- Kitaev, N., Kaiser, L., Levskaya, A., 2020. Reformer: The efficient Transformer. <https://doi.org/10.48550/arXiv.2001.04451>.

- Li, J., Li, D., Savarese, S., & Hoi, S. (2023). BLIP-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of the 40th international conference on machine learning. Presented at the international conference on machine learning, PMLR* (pp. 19730–19742).
- Li, Z., Deldari, S., Chen, L., Xue, H., Salim, F.D., 2024. SensorLLM: Aligning large language models with motion sensors for human activity recognition. <https://doi.org/10.48550/arXiv.2410.10624>.
- Lindemann, B., Müller, T., Vietz, H., Jazdi, N., & Weyrich, M. (2021). A survey on long short-term memory networks for time series prediction. *Procedia CIRP*, 99, 650–655. <https://doi.org/10.1016/j.procir.2021.03.088>
- Liu, C., Wan, Z., Cheng, S., Zhang, M., Arcucci, R., 2023. ETP: Learning transferable ECG representations via ECG-text pre-training. <https://doi.org/10.48550/arXiv.2309.07145>.
- Liu, C., Wan, Z., Ouyang, C., Shah, A., Bai, W., Arcucci, R., 2024. Zero-shot ECG classification with multimodal learning and test-time clinical knowledge enhancement. <https://doi.org/10.48550/arXiv.2403.06659>.
- Liu, R., Li, C., Tang, H., Ge, Y., Shan, Y., Li, G., 2024. ST-LLM: Large language models are effective temporal learners. <https://doi.org/10.48550/arXiv.2404.00308>.
- Liu, Y., Hu, T., Zhang, H., Wu, H., Wang, S., Ma, L., Long, M., 2024a. iTransformer: Inverted transformers are effective for time series forecasting. <https://doi.org/10.48550/arXiv.2310.06625>.
- Liu, Y., Wu, H., Wang, J., Long, M., 2023. Non-stationary transformers: Exploring the stationarity in time series forecasting. <https://doi.org/10.48550/arXiv.2205.14415>.
- Ma, Q., Liu, Z., Zheng, Z., Huang, Z., Zhu, S., Yu, Z., & Kwok, J. T. (2024). A survey on time-series pre-trained models. *IEEE Transactions on Knowledge and Data Engineering*, 36, 7536–7555. <https://doi.org/10.1109/TKDE.2024.3475809>
- Middlehurst, M., Schäfer, P., & Bagnall, A. (2024). Bake off redux: A review and experimental evaluation of recent time series classification algorithms. *IEEE Transactions on Knowledge and Data Engineering*, 38, 1958–2031. <https://doi.org/10.1007/s10618-024-01022-1>
- Mohammadi Foumani, N., Miller, L., Tan, C. W., Webb, G. I., Forestier, G., & Salehi, M. (2024). Deep Learning for time series classification and extrinsic regression: A current survey. *ACM Computing Surveys*, 56, 1–45. <https://doi.org/10.1145/3649448>
- Mudelsee, M. (2019). Trend analysis of climate time series: A review of methods. *Earth-Science Reviews*, 190, 310–322. <https://doi.org/10.1016/j.earscirev.2018.12.005>
- Nie, Y., Nguyen, N.H., Sinthong, P., Kalagnanam, J., 2023. A time series is worth 64 words: Long-term forecasting with transformers. <https://doi.org/10.48550/arXiv.2211.14730>.
- Qian, L., Zuo, Q., Li, D., & Zhu, H. (2025). DGMSCl: A dynamic graph mixed supervised contrastive learning approach for class imbalanced multivariate time series classification. *Neural Networks : The official Journal of the International Neural Network Society*, 185, Article 107131. <https://doi.org/10.1016/j.neunet.2025.107131>
- Qiu, J., Han, W., Zhu, J., Xu, M., Rosenberg, M., Liu, E., Weber, D., Zhao, D., 2023. Transfer knowledge from natural language to electrocardiography: Can we detect cardiovascular disease through language models? <https://doi.org/10.48550/arXiv.2301.09017>.
- Radford, A., Kim, J.W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I., 2021. Learning transferable visual models from natural language supervision. <https://doi.org/10.48550/arXiv.2210.00020>.
- Sun, C., Li, H., Li, Y., Hong, S., 2024. TEST: Text prototype aligned embedding to activate LLM's ability for time series. <https://doi.org/10.48550/arXiv.2308.08241>.
- Tan, C.W., Dempster, A., Bergmeir, C., Webb, G.I., 2022. MultiRocket: Multiple pooling operators and transformations for fast and effective time series classification. <https://doi.org/10.48550/arXiv.2102.00457>.
- Tao, X., Pan, T., Cheng, M., Luo, Y., 2024. Hierarchical multimodal LLMs with semantic space alignment for enhanced time series classification. <https://doi.org/10.48550/arXiv.2410.18686>.
- Wagner, P., Strodthoff, N., Boussejot, R.-D., Kreisler, D., Lunze, F. I., Samek, W., & Schaeffter, T. (2020). PTB-XL, a large publicly available electrocardiography dataset. *Scientific Data*, 7, 154. <https://doi.org/10.1038/s41597-020-0495-6>
- Wang, C., Qi, Q., Wang, J., Sun, H., Zhuang, Z., Wu, J., Zhang, L., Liao, J., 2024. ChatTime: A unified multimodal time series foundation model bridging numerical and textual data. <https://doi.org/10.48550/arXiv.2412.11376>.
- Wang, J., Cheng, M., Mao, Q., Liu, Q., Xu, F., Li, X., Chen, E., 2024. TableTime: Reformulating time series classification as zero-shot table understanding via large language models. <https://doi.org/10.48550/arXiv.2411.15737>.
- Wang, P., Zheng, H., Xu, Q., Dai, S., Wang, Y., Yue, W., Zhu, W., Qian, T., & Zhao, L. (2025). TS-HTFA: Advancing time-series forecasting via hierarchical text-free alignment with large language models. *Symmetry*, 17, 401. <https://doi.org/10.3390/sym17030401>
- Wang, S., Zhang, T., Li, Y., Li, P., Wu, H., & Li, K. (2024c). Continuous hand gestures detection and recognition in emergency Human-robot interaction based on the inertial measurement unit. *IEEE Transactions on Instrumentation and Measurement*, 73, 1–15. <https://doi.org/10.1109/TIM.2024.3440381>
- Wei, J., Tay, Y., Bommasani, R., Raffel, C., Zoph, B., Borgeaud, S., Yogatama, D., Bosma, M., Zhou, D., Metzler, D., Chi, E.H., Hashimoto, T., Vinyals, O., Liang, P., Dean, J., Fedus, W., 2022. Emergent abilities of large language models. <https://doi.org/10.48550/arXiv.2206.07682>.
- Wen, Q., Zhou, T., Zhang, C., Chen, W., Ma, Z., Yan, J., Sun, L., 2022. Transformers in Time Series: A survey. <https://doi.org/10.48550/arXiv.2202.07125>.
- Wu, H., Hu, T., Liu, Y., Zhou, H., Wang, J., & Long, M. (2023). TimesNet: Temporal 2D-variation modeling for. *General Time Series Analysis*. <https://doi.org/10.48550/arXiv.2210.02186>
- Yang, X., Zhang, Z., & Cui, R. (2022). TimeCLR: A self-supervised contrastive learning framework for univariate time series representation. *Knowledge-Based Systems*, 245, Article 108606. <https://doi.org/10.1016/j.knsys.2022.108606>
- Yosinski, J., Clune, J., Bengio, Y., Lipson, H., 2014. How transferable are features in deep neural networks? <https://doi.org/10.48550/arXiv.1411.1792>.
- Yue, Z., Wang, Y., Duan, J., Yang, T., Huang, C., Tong, Y., & Xu, B. (2022). TS2Vec: Towards universal representation of time series. In , 36. *Proceedings of the AAAI conference on artificial intelligence* (pp. 8980–8987). <https://doi.org/10.1609/aaai.v36i8.20881>
- Zhang, B., Gao, J., & Yuan, Y. (2024a). Center-enhanced video captioning model with multimodal semantic alignment. *Neural networks : The official journal of the International Neural Network Society*, 180, Article 106744. <https://doi.org/10.1016/j.neunet.2024.106744>
- Zhang, X., Chowdhury, R.R., Gupta, R.K., Shang, J., 2024. Large language models for time series: A survey. <https://doi.org/10.48550/arXiv.2402.01801>.
- Zhao, B., Lu, H., Chen, S., Liu, J., & Wu, D. (2017). Convolutional neural networks for time series classification. *Journal of Systems Engineering and ElectronicsOpen Access*, 28, 162–169. <https://doi.org/10.21629/JSEE.2017.01.18>
- Zheng, K., Yu, S., & Chen, B. (2024). CI-GNN: A Granger causality-inspired graph neural network for interpretable brain network-based psychiatric diagnosis. *Neural Networks : The Official Journal of the International Neural Network Society*, 172, Article 106147. <https://doi.org/10.1016/j.neunet.2024.106147>
- Zhou, H., Zhang, S., Peng, J., Zhang, S., Li, J., Xiong, H., & Zhang, W. (2021). Informer: Beyond efficient transformer for long sequence time-series forecasting. In , 35. *Proceedings of the AAAI conference on artificial intelligence* (pp. 11106–11115). <https://doi.org/10.1609/aaai.v35i12.17325>
- Zhou, T., Niu, P., Wang, X., Sun, L., Jin, R., 2023. One fits all: Power General time series analysis by pretrained LM. <https://doi.org/10.48550/arXiv.2302.11939>.
- Zhou, Y., Yang, J., Zou, H., Xie, L., 2023. TENT: Connect language models with IoT sensors for zero-shot activity recognition. <https://doi.org/10.48550/arXiv.2311.08245>.